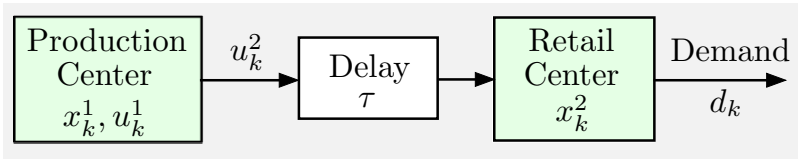

Figure 2.8.2. Illustration of a simple supply chain system for Example 2.8.1.

u_k^1: The amount produced at time k.

u_k^2: The amount shipped at time k (and arriving at the retail center τ time units later).

The state at time k is the stock available at the production and retail centers, x_k^1, x_k^2, plus the stock amounts that are in transit and have not yet arrived at the retail center $u_{k-\tau-1}^2, \ldots, u_{k-1}^2$. The control $u_k = (u_k^1, u_k^2)$ is chosen from some constraint set that may depend on the current state, and is subject to production capacity and transport availability constraints. The system equation is

$$x_{k+1}^1 = x_k^1 + u_k^1 - u_k^2, \qquad x_{k+1}^2 = x_k^2 + u_{k-\tau}^2 - d_k,$$

and involves the delayed control component $u_{k-\tau}^2$. Thus the exact DP algorithm involves state augmentation as introduced in Section 1.6.5, and may thus be much more complicated than in the case where there are no delays.†

The cost at time k consists of three components: a production cost that depends on x_k^1 and u_k^1, a transportation cost that depends on u_k^2, and a fulfillment cost that depends on x_k^2 [which includes positive costs for both excess inventory (i.e., $x_k^2 > d_k$) and for backordered demand (i.e., $x_k^2 < d_k$)]. The precise forms of these cost components are immaterial for the purposes of this example.

Here the control vector u_k is often continuous (or a mixture of discrete and continuous components), so it may be essential for the purposes of rollout to use the continuous optimization framework of this section. In particular, at the current stage k, we know the current state, which includes x_k^1, x_k^2, and the amounts of stock in transit together with their scheduled arrival times at the retail center. We then apply some heuristic optimization to determine the stream of future production and shipment levels over ℓ steps, and use the first component of this stream as the control applied by rollout. As an example we may use as base policy one that brings the retail inventory to some target value ℓ stages ahead, and possibly keep it at that value for a portion of the remaining periods. This is a nonlinear programming or mixed integer programming problem that may be solvable with available software far more efficiently than by a discretized form of DP.

† Despite the fact that with large delays, the size of the augmented state space can become very large (cf. Section 1.6.5), the implementation of rollout schemes is not affected much by this increase in size. For this reason, rollout can be very well suited for problems involving delayed effects of past states and controls.

A major benefit of rollout in the supply chain context is that it can readily incorporate on-line replanning. This is necessary when unexpected demand changes, production or transport equipment failures occur, or updated forecasts become available.

The following example deals with a common class of problems of resource allocation over time.

Example 2.8.2 (Multistage Linear and Mixed Integer Programming)

Let us consider a deterministic optimal control problem with linear system equation

$$x_{k+1} = A_k x_k + B_k u_k + d_k, \qquad k = 0, \ldots, N-1,$$

where A_k and B_k are known matrices of appropriate dimension, d_k is a known vector, and x_k and u_k are column vectors. The cost function is linear of the form

$$c_N{}' x_N + \sum_{k=0}^{N-1} (c_k{}' x_k + d_k{}' u_k),$$

where c_k and d_k are known column vectors of appropriate dimension, and a prime denotes transpose. The terminal state and state-control pairs (x_k, u_k) are constrained by

$$x_N \in T, \qquad (x_k, u_k) \in P_k, \qquad k = 0, \ldots, N-1,$$

where T and P_k, $k = 0, \ldots, N-1$, are given sets, which are specified by linear and possibly integer constraints.

As an example, consider a multi-item production system, where the state is $x_k = (x_k^1, \ldots, x_k^n)$ and x_k^i represents stock of item i available at the start of period k. The state evolves according to the system equation

$$x_{k+1}^i = \sum_{j=1}^n a_k^{ij} u_k^{ij} - d_k^i, \qquad i = 1, \ldots, n,$$

where u_k^{ij} is the amount of product i that is used during time k for the manufacture of product j, a_k^{ij} are known scalars that are related to the underlying production process, and d_k^i is a deterministic demand of product i that is fulfilled at time k. One constraint here is that

$$\sum_{j=1}^n u_k^{ij} \leq x_k^i, \qquad i = 1, \ldots, n,$$

and there are additional linear and integer constraints on (x_k, u_k), which are collected in a general constraint of the form $(x_k, u_k) \in P_k$ (e.g., nonnegativity, production capacity, storage constraints, etc). Note that the problem

may be further complicated by production delays, as in the preceding supply chain Example 2.8.1. Moreover, while in this section we focus on deterministic problems, we may envision a stochastic version of the problem where the demands d_k^i are random with given probability distributions, which are subject to revisions based on randomly received forecasts.

The problem may be solved using a linear or mixed integer programming algorithm, but this may be very time-consuming when N is large. Moreover, the problem will need to be resolved on-line if some of the problem data changes and replanning is necessary. A suboptimal alternative is to use truncated rollout with an ℓ-stage mixed integer optimization, and a polyhedral terminal cost function $\tilde{J}_{k+\ell}$ to provide a terminal cost optimization. A simple possibility is no terminal cost $[\tilde{J}_{k+\ell}(x_{k+\ell}) \equiv 0]$, and another possibility is a polyhedral lower bound approximation that can be based on relaxing the integer constraints after stage $k+\ell$, or some kind of training approach that uses data.

We will next discuss how rollout can accommodate stochastic disturbances by using deterministic optimization ideas based on certainty equivalence and the methodology of stochastic programming.

2.8.2 Rollout Based on Stochastic Programming

We have focused so far in this section on rollout that relies on deterministic continuous optimization. There is an important class of methods, known as *stochastic programming*, which can be used for stochastic optimal control, but bears a close connection to continuous spaces deterministic optimization. We will first describe this connection for two-stage problems, then discuss extensions to many-stages problems, and finally show how rollout can be brought to bear for their approximate solution.

Example 2.8.3 (Two-Stage Stochastic Programming)

Consider a stochastic problem of optimal decision making over two stages: In the first stage we will choose a finite-dimensional vector u_0 from a subset U_0 with cost $g_0(u_0)$. Then an uncertain event represented by a random variable w_0 will occur, whereby w_0 will take one of the values $w^1, \ldots, w^m$ with corresponding probabilities $p^1, \ldots, p^m$. Once w_0 occurs, we will know its value w^i, and we must then choose at the second stage a vector u_1^i from a subset $U_1(u_0, w^i)$ at a cost $g_1(u_1^i, w^i)$. The objective is to minimize the expected cost

$$g_0(u_0) + \sum_{i=1}^{m} p^i g_1(u_1^i, w^i),$$

subject to

$$u_0 \in U_0, \qquad u_1^i \in U_1(u_0, w^i), \quad i = 1, \ldots, m.$$

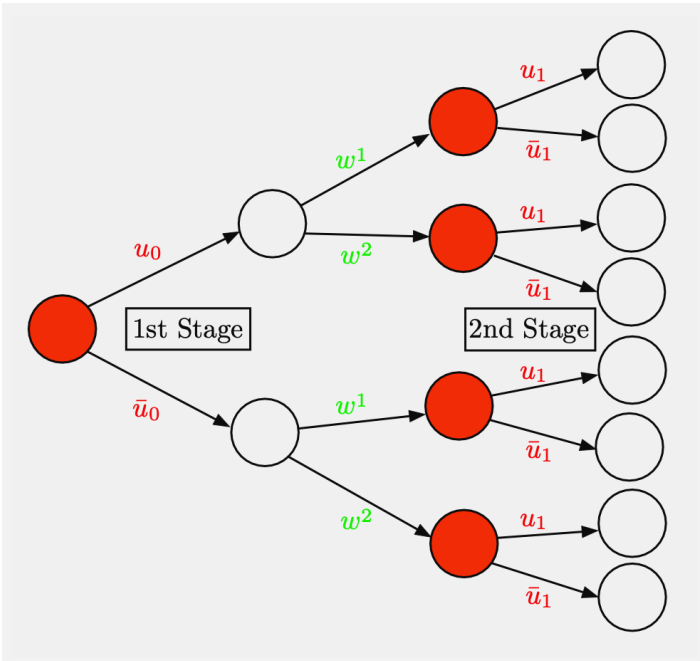

Figure 2.8.3. Illustration of the DP problem associated with two-stage stochastic programming; cf. Example 2.8.3. The figure depicts the case where each variable u_0, w_0, and u_1 can take only two values. A similar conversion to a DP problem is possible for a multistage stochastic programming problem, involving multiple choices of decisions, each followed by an uncertain event whose outcome is perfectly observed by the decision maker.

We can view this problem as a two-stage DP problem, where $x_1 = w_0$ is the system equation, the disturbance w_0 can take the values $w^1, \ldots, w^m$ with probabilities $p^1, \ldots, p^m$, the cost of the first stage is $g_0(u_0)$, the cost of the second stage is $g_1(x_1, u_1)$, and the terminal cost is 0. The intuitive meaning is that since at time 0 we don't know yet which of the m values w^i of w_0 will occur, we must calculate (in addition to u_0) a separate second stage decision u_1^i for each i, which will be used after we know that the value of w_0 is w^i.

However, if u_0 and u_1 take values in a continuous space such as the Euclidean spaces $\Re^{d_0}$ and $\Re^{d_1}$, respectively, we can also equivalently view the problem as a nonlinear programming problem of dimension $(d_0 + md_1)$ (the optimization variables are u_0 and u_1^i, $i = 1, \ldots, m$).

For a generalization of the preceding example, consider the stochastic DP problem of Section 1.3 for the case where there are only two stages, and the disturbances w_0 and w_1 can independently take one of the m values $w^1, \ldots, w^m$ with corresponding probabilities $p_0^1, \ldots, p_0^m$ and $p_1^1, \ldots, p_1^m$, respectively. The optimal cost function $J_0(x_0)$ is given by the two-stage

DP algorithm

$$J_0(x_0) = \min_{u_0 \in U_0(x_0)} \Bigg[\sum_{i=1}^{m} p_0^i \Bigg\{ g_0(x_0, u_0, w^i) + \min_{u_1^i \in U_1(f_0(x_0,u_0,w^i))} \Bigg[\sum_{j=1}^{m} p_1^j \Big\{ g_1\big(f_0(x_0, u_0, w^i), u_1^i, w^j\big) + g_2\big(f_1\big(f_0(x_0, u_0, w^i), u_1^i, w^j\big)\big) \Big\} \Bigg] \Bigg\} \Bigg].$$

By bringing the inner minimization outside the inner brackets, we see that this DP algorithm is equivalent to solving the nonlinear programming problem

$$\begin{aligned} &\text{minimize} \quad \sum_{i=1}^{m} p_0^i \Bigg\{ g_0(x_0, u_0, w^i) + \sum_{j=1}^{m} p_1^j \Big\{ g_1\big(f_0(x_0, u_0, w^i), u_1^i, w^j\big) \\ &\qquad\qquad + g_2\big(f_1\big(f_0(x_0, u_0, w^i), u_1^i, w^j\big)\big) \Big\} \Bigg\} \\ &\text{subject to } u_0 \in U_0(x_0), \qquad u_1^i \in U_1\big(f_0(x_0, u_0, w^i)\big), \quad i = 1, \ldots, m. \end{aligned} \tag{2.83}$$

If the controls u_0 and u_1^i are elements of $\Re^d$, this problem involves $d(1 + m)$ scalar variables. An example is the multi-item production problem described in Example 2.8.2 in the case where the demands w_k^i and/or the production coefficients a_k^{ij} are stochastic.

We can also consider an N-stage stochastic optimal control problem. A similar reformulation as a nonlinear programming problem is possible. It converts the N-stage stochastic problem into a deterministic optimization problem of dimension that grows exponentially with the number of stages N. In particular, for an N-stage problem, the number of control variables expands by a factor m with each additional stage. The total number of variables is bounded by

$$d\big(1 + m + m^2 + \cdots + m^{N-1}\big),$$

where m is the maximum number of values that a disturbance can take at each stage and d is the dimension of the control vector.

2.8.3 Stochastic Rollout with Certainty Equivalence

The dimension of the preceding nonlinear programming formulation of the multistage stochastic optimal control problem with continuous control spaces can be very large. This motivates a variant of a rollout algorithm

that relies on a stochastic optimization for the current stage, and a deterministic optimization that relies on (assumed) certainty equivalence for the remaining stages, where the base policy is used. In this way, the dimension of the nonlinear programming problem to be solved by rollout is drastically reduced.

This rollout algorithm operates as follows: Given a state x_k and control $u_k \in U_k(x_k)$, we consider the next states x_{k+1}^i that correspond to the m possible values w_k^i, $i = 1, \ldots, m$, which occur with the known probabilities p_k^i, $i = 1, \ldots, m$. We then consider the approximate Q-factors

$$\tilde{Q}_k(x_k, u_k) = \sum_{i=1}^{m} p_k^i \big(g_k(x_k, u_k, w_k^i) + \tilde{H}_{k+1}(x_{k+1}^i)\big), \tag{2.84}$$

where $\tilde{H}_{k+1}(x_{k+1}^i)$ is the cost of a base policy, which starting at stage $k+1$ from

$$x_{k+1}^i = f_k(x_k, u_k, w_k^i),$$

optimizes the cost-to-go starting from x_{k+1}^i, while assuming that the future disturbances $w_{k+1}, \ldots, w_{N-1}$, will take some nominal (nonrandom) values $\overline{w}_{k+1}, \ldots, \overline{w}_{N-1}$. The rollout control $\tilde{\mu}_k(x_k)$ computed by this algorithm is

$$\tilde{\mu}_k(x_k) \in \arg \min_{u_k \in U_k(x_k)} \tilde{Q}_k(x_k, u_k). \tag{2.85}$$

Note that this rollout algorithm does not have the cost improvement property, because it involves an approximation: the cost $\tilde{H}_{k+1}(x_{k+1}^i)$ used in Eq. (2.84) is an approximation to the cost of a policy. It is the cost of a policy applied to the certainty equivalent version of the original stochastic problem.

The key fact now is that the problem (2.85) can be viewed as a seamless $(N-k)$-stage deterministic optimization, which involves the control u_0, and for each value w_k^i of the disturbance w_k, the sequence of controls $(u_{k+1}^i, \ldots, u_{N-1}^i)$. If the controls are elements of $\Re^d$, this deterministic optimization involves a total of

$$d\big(1 + (N-k-1)m\big) \tag{2.86}$$

scalar variables. Currently available deterministic optimization software can deal with quite large numbers of variables, particularly in the context of linear programming, so by using rollout in combination with certainty equivalence, very large problems with continuous state and control variables may be addressed. We refer to the paper by Hu et al. [HWP22] for an application of this idea to problems of maintenance scheduling.

Another possibility is to use multistep lookahead that aims to represent better the stochastic character of the uncertainty. Here at state x_k we solve an $(N-k)$-stage optimal control problem, where the uncertainty

is fully taken into account in the first ℓ stages, similar to stochastic programming, and in the remaining $N-k-\ell$ stages, the uncertainty is dealt with by certainty equivalence, by fixing the disturbances $w_{k+\ell},\ldots,w_{N-1}$ at some nominal values (we assume here for simplicity that $\ell < N-k$). If the controls are elements of $\Re^d$, and the number of values that the disturbances $w_0,\ldots,w_{N-1}$ can take is m, the total number of control variables of this problem is

$$d\big(1+m+\cdots+m^{\ell-1}+(N-k-\ell)m^{\ell}\big),$$

[this is the ℓ-step lookahead generalization of the formula (2.86)]. Once the optimal policy $\{\tilde{u}_k,\tilde{\mu}_{k+1},\tilde{\mu}_{k+2},\ldots\}$ for this problem is obtained, the first control component $\tilde{u}_k$ is applied at x_k and the remaining components $\{\tilde{\mu}_{k+1},\tilde{\mu}_{k+2},\ldots\}$ are discarded. Note also that this multistep lookahead approach may be combined with the ideas of multiagent rollout, which will be discussed in the next section.

2.9 MULTIAGENT ROLLOUT

We will now consider a special structure of the control space, whereby the control u_k consists of m components, $u_k=(u_k^1,\ldots,u_k^m)$, with a separable control constraint structure $u_k^\ell \in U_k^\ell(x_k)$, $\ell=1,\ldots,m$. The control constraint set is the Cartesian product

$$U_k(x_k)=U_k^1(x_k)\times\cdots\times U_k^m(x_k). \tag{2.87}$$

Conceptually, each component u_k^ℓ, $\ell=1,\ldots,m$, is chosen at stage k by a separate "agent" (a decision making entity), and for the sake of the following discussion, we assume that each set $U_k^\ell(x_k)$ is finite. We discussed this type of problem briefly in Section 1.6.7, and we will discuss it in this section in greater detail.

The one-step lookahead minimization

$$\tilde{u}_k \in \arg\min_{u_k\in U_k(x_k)} E\Big\{g_k(x_k,u_k,w_k)+J_{k+1,\pi}\big(f_k(x_k,u_k,w_k)\big)\Big\}, \tag{2.88}$$

where π is a base policy, involves as many as n^m Q-factors, where n is the maximum number of elements of the sets $U_k^\ell(x_k)$ [so that n^m is an upper bound to the number of controls in $U_k(x_k)$, in view of the Cartesian product structure (2.87)]. As a result, the standard rollout algorithm requires an exponential [order $O(n^m)$] number of base policy cost computations per stage, which can be overwhelming even for moderate values of m.

This motivates an alternative and more efficient rollout algorithm, called *multiagent rollout* also referred to as *agent-by-agent rollout*, that still achieves the cost improvement property

$$J_{k,\tilde{\pi}}(x_k)\le J_{k,\pi}(x_k), \qquad \forall\ x_k,k, \tag{2.89}$$

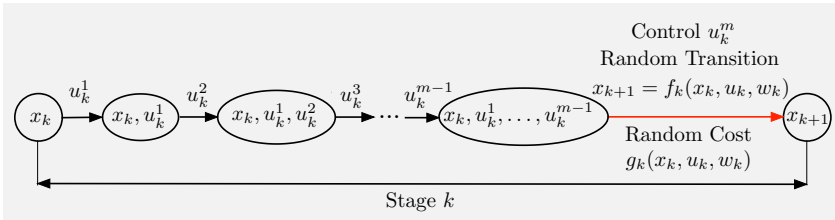

Figure 2.9.1 Equivalent formulation of the N-stage stochastic optimal control problem for the case where the control u_k consists of m components $u_k^1, u_k^2, \ldots, u_k^m$:

$$u_k = (u_k^1, \ldots, u_k^m) \in U_k^1(x_k) \times \cdots \times U_k^m(x_k);$$

cf. Section 1.6.7. The figure depicts the kth stage transitions. Starting from state x_k, we generate the intermediate states

$$(x_k, u_k^1), (x_k, u_k^1, u_k^2), \ldots, (x_k, u_k^1, \ldots, u_k^{m-1}),$$

using the respective controls $u_k^1, \ldots, u_k^{m-1}$. The final control u_k^m leads from $(x_k, u_k^1, \ldots, u_k^{m-1})$ to

$$x_{k+1} = f_k(x_k, u_k, w_k),$$

and a stage cost $g_k(x_k, u_k, w_k)$ is incurred. All of the preceding transitions, which involve the controls $u_k^1, \ldots, u_k^{m-1}$, incur zero cost.

where $J_{k,\tilde{\pi}}(x_k)$, $k = 0, \ldots, N$, is the cost-to-go of the rollout policy $\tilde{\pi}$ starting from state x_k. Indeed we will exploit the multiagent structure to construct an algorithm that maintains the cost improvement property at much smaller computational cost, namely requiring order $O(nm)$ base policy cost computations per stage.

A key idea here is that the computational requirements of the rollout one-step minimization (2.88) are proportional to the size of the control space and are independent of the size of the state space. We consequently reformulate the problem so that control space complexity is traded off with state space complexity, as discussed in Section 1.6.7. This is done by "unfolding" the control u_k into its m components $u_k^1, u_k^2, \ldots, u_k^m$. At the same time, between x_k and the next state $x_{k+1} = f_k(x_k, u_k, w_k)$, we introduce artificial intermediate "states" and corresponding transitions; see Fig. 2.9.1, given in Section 1.6.7 and repeated here for convenience.

It can be seen that this reformulated problem is equivalent to the original, since any control choice that is possible in one problem is also possible in the other problem, while the cost structure of the two problems is the same: each policy of the reformulated problem corresponds to a policy of the original problem, with the same cost function, and reversely.†

† A fine point here is that policies of the original problem involve functions

Consider now the standard rollout algorithm applied to the reformulated problem of Fig. 2.9.1, with a given base policy $\pi = \{\mu_0, \ldots, \mu_{N-1}\}$, which is also a policy of the original problem [so that $\mu_k = (\mu_k^1, \ldots, \mu_k^m)$, with each μ_k^ℓ, $\ell = 1, \ldots, m$, being a function of just x_k]. The algorithm involves a minimization over only one control component at the states x_k and at the intermediate states

$$(x_k, u_k^1), (x_k, u_k^1, u_k^2), \ldots, (x_k, u_k^1, \ldots, u_k^{m-1}).$$

In particular, *for each stage k, the algorithm requires a sequence of m minimizations, once over each of the agent controls $u_k^1, \ldots, u_k^m$, with the past controls determined by the rollout policy, and the future controls determined by the base policy.* Assuming a maximum of n elements in the constraint sets $U_k^\ell(x_k)$, the computation required at each stage k is of order $O(n)$ for each of the "states"

$$x_k, (x_k, u_k^1), \ldots, (x_k, u_k^1, \ldots, u_k^{m-1}),$$

for a total of order $O(nm)$ computation.

To elaborate, at $(x_k, u_k^1, \ldots, u_k^{\ell-1})$ with $\ell \leq m$, and for each of the controls $u_k^\ell \in U_k^\ell(x_k)$, we generate by simulation a number of system trajectories up to stage N, with all future controls determined by the base policy. We average the costs of these trajectories, thereby obtaining the Q-factors corresponding to $(x_k, u_k^1, \ldots, u_k^{\ell-1}, u_k^\ell)$, for all values $u_k^\ell \in U_k^\ell(x_k)$ (with the preceding controls $u_k^1, \ldots, u_k^{\ell-1}$ held at the values computed earlier, and the future controls $u_k^{\ell+1}, \ldots, u_k^m, u_{k+1}, \ldots, u_{N-1}$ determined by the base policy). We then select the control $u_k^\ell \in U_k^\ell(x_k)$ that corresponds to the minimal Q-factor.

Prerequisite assumptions for the preceding algorithm to work in an on-line multiagent setting are:

(a) All agents have access to the current state x_k as well as the base policy (including the control functions μ_n^ℓ, $\ell = 1, \ldots, m$, $n = 0, \ldots, N-1$ of all agents).

(b) There is an order in which agents compute and apply their local controls.

(c) The agents share their information, so agent ℓ knows the local controls $u_k^1, \ldots, u_k^{\ell-1}$ computed by the predecessor agents $1, \ldots, \ell-1$ in the given order.

---

of x_k, while policies of the reformulated problem involve functions of the choices of the preceding agents, as well as x_k. However, by successive substitution of the control functions of the preceding agents, we can view control functions of each agent as depending exclusively on x_k. It follows that the multi-transition structure of the reformulated problem cannot be exploited to reduce the cost function beyond what can be achieved with a single-transition structure.

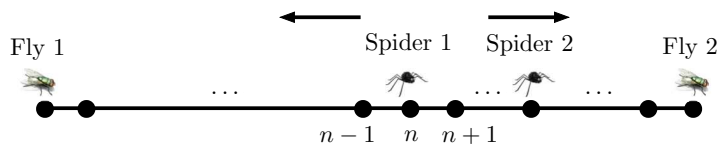

Figure 2.9.2 Illustration of the two-spiders and two-flies problem. The spiders move along integer points of a line. The two flies stay still at some integer locations. The character of the optimal policy is to move the two spiders towards two different flies.

Multiagent rollout with the given base policy starts with spider 1 at location n, and calculates the two Q-factors of moving to locations $n-1$ and $n+1$, assuming that the remaining moves of the two spiders will be made using the go-towards-the-nearest-fly base policy. The Q-factor of going to $n-1$ is smallest because it saves in unnecessary moves of spider 1 towards fly 2, so spider 1 will move towards fly 1. The trajectory generated by multiagent rollout is to move spiders 1 and 2 towards flies 1 and 2, respectively, then spider 2 first captures fly 2, and then spider 1 captures fly 1.

Note that the rollout policy obtained from the reformulated problem may be different from the rollout policy obtained from the original problem. However, the former rollout algorithm is far more efficient than the latter in terms of required computation, while still maintaining the cost improvement property (2.89).

Illustrative Examples

The following spiders-and-flies example illustrates how multiagent rollout may exhibit intelligence and agent coordination that is totally lacking from the base policy. This behavior has been supported by computational experiments and analysis with larger (two-dimensional) spiders-and-flies problems.

Example 2.9.1 (Spiders and Flies)

We have two spiders and two flies moving along integer locations on a straight line. For simplicity we assume that the flies' positions are fixed at some integer locations, although the problem is qualitatively similar when the flies move randomly. The spiders have the option of moving either left or right by one unit; see Fig. 2.9.2. The objective is to minimize the time to capture both flies. The problem has essentially a finite horizon since the spiders can force the capture of the flies within a known number of steps.

The salient feature of the optimal policy here is to move the two spiders towards different flies. The minimal time to capture is the maximum of the initial distances of the two spider-fly pairs of the optimal policy.

Let us apply multiagent rollout with the base policy that directs each spider to move one unit towards the closest fly position (a tie is broken by moving towards the right-side fly). The base policy is poor because it may unnecessarily move both spiders in the same direction, when in fact only one

is needed to capture the fly. This limitation is due to the lack of coordination between the spiders: each acts selfishly, ignoring the presence of the other. We will see that rollout restores a significant degree of coordination between the spiders through an optimization that takes into account the long-term consequences of the spider moves.

According to the multiagent rollout mechanism, the spiders choose their moves one-at-a-time, optimizing over the two Q-factors corresponding to the right and left moves, while assuming that future moves will be chosen according to the base policy. Let us consider a stage, where the two flies are alive, while both spiders are closest to fly 2, as in Fig. 2.9.2. Then the rollout algorithm will start with spider 1 and calculate two Q-factors corresponding to the right and left moves, while using the base heuristic to obtain the next move of spider 2, and the remaining moves of the two spiders. Depending on the values of the two Q-factors, spider 1 will move to the right or to the left, and it can be seen that it will choose to *move away from spider 2* even if doing so increases its distance to its closest fly *contrary to what the base heuristic will do*. Then spider 2 will act similarly and the process will continue. Intuitively, at the state of Fig. 2.9.2, spider 1 moves away from spider 2 and fly 2, because it recognizes that spider 2 will capture earlier fly 2, so it might as well move towards the other fly.

Thus *the multiagent rollout algorithm induces implicit move coordination*, i.e., each spider moves in a way that takes into account future moves of the other spider. In fact it can be verified that the algorithm will produce an optimal sequence of moves starting from any initial spider positions. It can also be seen that ordinary rollout (both flies move at once) will also produce an optimal move sequence.

The example illustrates how a poor base heuristic can produce an excellent rollout solution, something that can be observed frequently in many other problems. Intuitively, the key fact is that rollout is "farsighted" in the sense that it can benefit from control calculations that reach far into future stages.

A two-dimensional generalization of the example is also interesting. Here the flies are at two corners of a square in the plane. It can be shown that the two spiders, starting from the same position within the square, will separate under the rollout policy, with each moving towards a different spider, while under the base policy, they will move in unison along the shortest path to the closest surviving fly. Again this will happen for both standard and multiagent rollout.

Let us consider another example of a discrete optimization problem that can be solved efficiently with multiagent rollout.

Example 2.9.2 (Multi-Vehicle Routing)

Consider a multi-vehicle routing problem, whereby m vehicles move along the arcs of a given graph, aiming to perform tasks located at the nodes of the graph; see Fig. 2.9.3. When a vehicle reaches a task, it performs it, and can move on to perform another task. We wish to perform the tasks in a minimum number of individual vehicle moves.

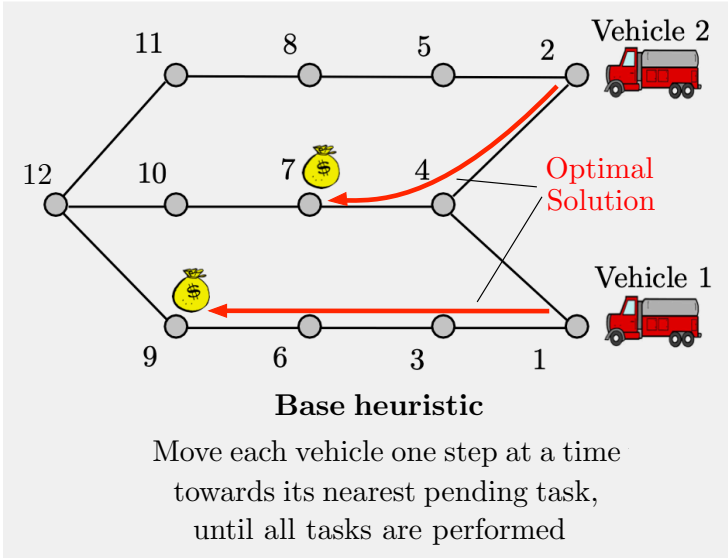

Figure 2.9.3 An instance of the vehicle routing problem of Example 2.9.2, and the multiagent rollout approach. The two vehicles aim to collectively perform the two tasks as fast as possible. Here, we should avoid sending both vehicles to node 4, towards the task at node 7; sending only vehicle 2 towards that task, while sending vehicle 1 towards the task at node 9 is clearly optimal. However, the base heuristic has "limited vision" and does not perceive this. By contrast the standard and the one-vehicle-at-a-time rollout algorithms look beyond the first move and avoid this inefficiency: they examine both moves of vehicle 1 to nodes 3 and 4, and use the base heuristic to explore the corresponding trajectories to the end of the horizon, and discover that vehicle 2 can reach quickly node 7, and that it is best to send vehicle 1 towards node 9.

In particular, the one-vehicle-at-a-time rollout algorithm will operate as follows: given the starting position pair $(1, 2)$ of the vehicles and the current pending tasks at nodes 7 and 9, we first compare the Q-factors of the two possible moves of vehicle 1 (to nodes 3 and 4), assuming that all the remaining moves will be selected by the base heuristic at the beginning of each stage. Thus vehicle 1 will choose to move to node 3. Then with knowledge of the move of vehicle 1 from 1 to 3, we select the move of vehicle 2 by comparing the Q-factors of its two possible moves (to nodes 4 and 5), taking also into account the fact that the remaining moves will be made according to the base heuristic. Thus vehicle 2 will choose to move to node 4.

We then continue at the next state [vehicle positions at (3,4) and pending tasks at nodes 7 and 9], select the base heuristic moves of vehicles 1 and 2 on the path to the closest pending tasks [(9 and 7), respectively], etc. Eventually the rollout finds the optimal solution (move vehicle 1 to node 9 in three moves and move vehicle 2 to node 7 in two moves), which has a total cost of 5. By contrast it can be seen that the base heuristic at the initial state will move both vehicles to node 4 (towards the closest pending task), and generate a trajectory that moves vehicle 1 along the path $1 \to 4 \to 7$ and vehicle 2 along the path $2 \to 4 \to 7 \to 10 \to 12 \to 9$, while incurring a total cost of 7.